

 This is a copy of a chat between Claude and Zack. Content may include unverified or unsafe content that do not represent the views of Anthropic. Shared snapshot may contain attachments and data not displayed here.

I just shared Claude with my Tech Support team. Their job is to help customers that have technical trouble with their CNC machines. Problems can range anywhere between mechanical, electrical, software, or warranty issues. They answer phone calls (RingCentral) and ZenDesk (for digital support). I just had a few export a few transcripts and Claude created an amazing product guide. I would like to take this further.

Lets take this one step at a time. Tell me how I could integrate RingCentral transcripts (on the RX plan) and ZenDesk into Claude - or at least some main database we can query. Tell me things or creative solutions that i may not know

Great question — and you're at exactly the right inflection point. Here's a practical breakdown of your options, from easiest to most powerful:

Option 1: Manual Export (Where You Are Now)

You're already doing this. The ceiling is low, but it works for ad-hoc analysis. Good for occasional product guide generation, not for ongoing operations.

Option 2: ZenDesk → Claude via API (Low-Code, High Value)

ZenDesk has a robust REST API. You can:

- **Auto-export tickets** on a schedule (daily, weekly) as JSON or CSV and drop them into a shared folder that Claude can read from
- Use **ZenDesk Triggers + Webhooks** to push resolved tickets to a database in real time
- Connect ZenDesk directly to Claude via its MCP integration (which I can help

you set up here in Claude.ai — I was just checking if it's available)

The most practical "quick win" here: export resolved tickets as CSV → feed to Claude → ask it to identify the top 10 recurring CNC failure patterns this month.

Option 3: RingCentral Transcripts → Searchable Database

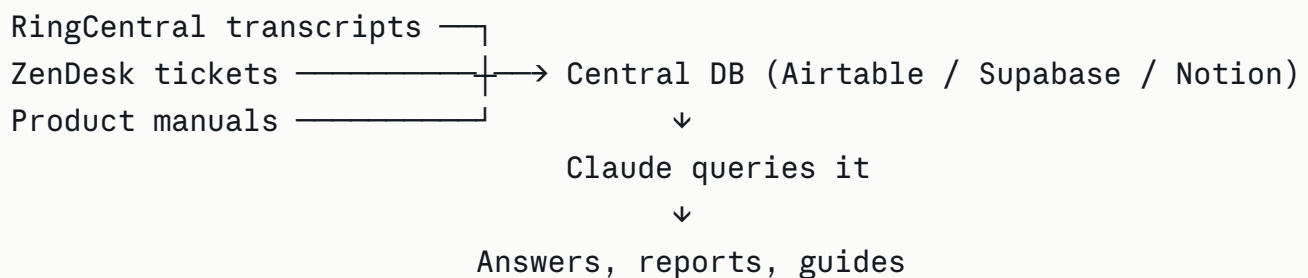
On the **RX plan**, you have access to AI-generated call transcripts and summaries. Here's what most people don't realize:

- **RingCentral has an API** that lets you pull transcripts programmatically
- You can pipe those transcripts into a **simple database** (even a Google Sheet or Notion database works at small scale) or something more serious like **Airtable, Supabase, or PostgreSQL**
- Once in a database, Claude can query it via MCP or API calls

The creative unlock: RingCentral RX also gives you call sentiment data and topic tagging — you can use those signals *before* Claude even reads the transcript.

Option 4: Unified Knowledge Base (The Real Prize)

This is what you're actually building toward. The architecture looks like:



What this unlocks that most people miss:

- Ask Claude "what's the most common failure on Model X in the last 90 days?" and get a real answer
- Auto-generate monthly support summaries for your engineering team
- Flag warranty patterns — e.g., if 8 customers report the same electrical fault

within 30 days of purchase, Claude surfaces that proactively

- Build an internal chatbot your techs can ask *during* a call: "Customer says the Z-axis is stuttering on a [Model Y] — what are the top 3 causes?"

Option 5: Claude Projects + Docs (No-Code, Right Now)

If you're not ready for API integrations, Claude's **Projects** feature lets you upload reference documents that persist. You could:

1. Do a weekly export of ZenDesk + RingCentral
2. Drop them into a Claude Project
3. Your whole team queries that shared context

It's manual, but it gives your team a surprisingly powerful shared brain with zero engineering.

My Recommendation for Your Team

Start with **ZenDesk → CSV export → Claude Project** this week. Then, if you want, I can help you build a lightweight automation that pulls from both ZenDesk and RingCentral automatically and feeds a central queryable database — no heavy IT lift required.

Want to go deeper on any of these paths? I can also help you draft the actual integration setup step by step.



I've turned on connector search. Please retry now.

Used 3 tools

Unfortunately neither ZenDesk nor RingCentral have native MCP connectors available in the registry yet — and same for common databases like Airtable or Notion.

That said, this doesn't block you. The most practical path forward is still very achievable:

What I'd suggest next:

1. **ZenDesk → Google Sheets** via ZenDesk's built-in export or a no-code tool like Zapier/Make — Google Sheets *does* have MCP connectors available in Claude, so once data lands there, Claude can query it directly
2. **RingCentral RX transcripts → Google Drive** — RingCentral can auto-save transcripts to a folder, and Claude can read from Drive
3. That gives you a living, queryable knowledge base your team can use today without any custom engineering

Want me to walk through setting up that Google Sheets pipeline? It's probably the fastest path to a real working system for your team. Or if you'd prefer to explore a more robust database approach (like Supabase or a simple backend), I can map that out too.

Ok, take me through Option 2 step by step

Let me make sure I point you in the right direction — let me check the current ZenDesk API docs approach before walking you through it.

Which part of Option 2 do you want to tackle first?

whatever of the 3 is automated and most robust

Great — that's **ZenDesk Webhooks → real-time push to database**. And good news: with your comfort level we can make this clean and manageable.

Here's the full picture of what we're building before we dive in:

ZenDesk ticket resolved



Step 1: Choose Your Database

This is the foundation everything else writes to. Two solid options:

	Google Sheets	Supabase
Setup time	5 min	20 min
Cost	Free	Free tier available
Scale	~50k rows comfortably	Millions of rows
Claude access	Native MCP connector	Via API
Best for	Getting started fast	Long-term, serious use

My recommendation for you right now: Google Sheets to start. You can migrate later. It lets you see and verify the data easily, your team can eyeball it, and Claude can connect to it natively.

Action: Create a new Google Sheet called `CNC Support Log` with these columns:

```
Ticket ID | Date | Channel | Agent | Customer |  
Issue Category | Machine Model | Resolution |  
Full Description | Tags | Ticket URL
```

Step 2: Set Up Make.com as Your Middleman

Make.com (formerly Integromat) is the glue between ZenDesk and your database. Free tier handles up to 1,000 operations/month — likely enough to start.

1. Go to **make.com** and create a free account
 2. Click **Create a new scenario**
 3. Search for and add **ZenDesk** as your trigger module
 4. Select trigger: **"Watch Tickets"** (fires when a ticket is updated/resolved)
 5. Connect your ZenDesk account — it'll ask for your ZenDesk subdomain + API token
 - In ZenDesk: go to **Admin → Apps & Integrations → APIs → Zendesk API** → generate a token
 6. Set the filter: only trigger when **Status = Solved**
-

Step 3: Map ZenDesk Fields → Your Google Sheet

Still in Make.com, add a second module:

1. Click the + after ZenDesk
 2. Search **Google Sheets** → select **"Add a Row"**
 3. Connect your Google account and point it to your **CNC Support Log** sheet
 4. Map the fields — Make shows you every ZenDesk field available. Key ones to grab:
 - **Ticket ID** → Ticket ID column
 - **Created At** → Date
 - **Assignee Name** → Agent
 - **Requester Name** → Customer
 - **Subject** + **Description** → Issue / Full Description
 - **Tags** → Tags
 - **Ticket URL** → Ticket URL
 5. Hit **Run Once** to test — you should see a row appear in your Sheet
-

Step 4: Add a Claude Enrichment Step (The Power Move)

This is where it gets interesting. *Before* writing to the Sheet, you can have Make call the Claude API to auto-categorize each ticket. Add a module between ZenDesk and Google Sheets:

1. Add an **HTTP module** in Make
2. Point it to <https://api.anthropic.com/v1/messages>
3. Send the ticket subject + description to Claude with a prompt like:

"You are a CNC support classifier. Given this support ticket, return JSON with: issue_category (Mechanical/Electrical/Software/Warranty), machine_model (extract if mentioned), severity (Low/Medium/High), and a one-sentence summary."

4. Parse Claude's JSON response and map those enriched fields into your Sheet columns

Now every ticket that gets resolved **automatically lands in your database, pre-categorized, summarized, and ready to query.**

Step 5: Turn It On

1. Back in Make, toggle your scenario to **ON**
 2. Set the schedule to run every 15 minutes (or real-time with a webhook — Make supports both)
 3. From this point forward, every resolved ZenDesk ticket flows in automatically
-

What You Can Do Once It's Running

With a live, growing database of categorized tickets, you (or your team) can ask Claude things like:

- *"What are the top 5 recurring issues on the [Model X] in the last 60 days?"*
- *"Which agent has the highest rate of warranty escalations?"*
- *"Summarize all Electrical issues from this month into a report for engineering"*
- *"Are there any patterns suggesting a batch defect in machines sold in Q1?"*

Want me to go deeper on any specific step — particularly the Claude enrichment call in Step 4, or how to connect Google Sheets to Claude once the data is flowing?

I am just curious. Is there a way an ai agent could be listening to a ringCentral call in real-time and prompt my techs with questions and potential solutions

Yes — and it's more accessible than most people realize. Here's the honest breakdown:

The Short Answer

RingCentral RX has a **Real-Time Transcription API** (called Live Transcription). It streams words as they're spoken via WebSocket. You can pipe that stream to an AI agent that listens, understands context, and pushes suggestions to your tech — all while the call is happening.

What It Would Look Like for Your Tech

They're on a call. Customer says:

"The Z-axis is making a grinding noise and it started after we changed the tooling"

Within a few seconds, a small sidebar panel on their screen shows:

💡 **Likely cause:** Improper tramming after tooling change, or loose ballscrew coupler

🔧 **Ask the customer:** "Did the noise start immediately or after a few cycles?"

📁 **Related tickets:** 3 similar cases last month — all resolved by re-tramming

No typing. No searching. Just a quiet co-pilot running in the background.

The Architecture

```
RingCentral call
  ↓
Live Transcription API (WebSocket stream)
  ↓
Your backend (Node.js or Python)
  ↓
Claude API ← with your CNC knowledge base + past tickets
  ↓
Push suggestion to tech's screen (simple web UI or browser extension)
```

The Key Pieces

1. RingCentral Live Transcription

Available on RX plan. Streams transcript chunks in real-time via WebSocket. You'd need a small backend to receive and buffer the stream.

2. Claude as the brain

Every 30 seconds or so (or on key trigger phrases), you send the last chunk of transcript + conversation history to Claude with a system prompt like:

"You are a CNC expert assistant. Listen to this support call and suggest the most likely cause, next diagnostic question, and any relevant past solutions. Be brief."

3. The display layer

The simplest version is just a web page your techs keep open alongside their softphone. More polished: a Chrome extension that sits next to RingCentral's web app.

What Makes This Genuinely Powerful

- You can feed it your **entire ticket history** as context — so it's learning from every past case
- It can detect **warranty trigger phrases** ("it's only 3 months old") and

automatically flag it

- It can **auto-draft the ZenDesk ticket** in the background while the call is happening, ready to submit when they hang up
- It can tell the tech when to **escalate** based on patterns it recognizes

Honest Caveats

- This requires a small backend — a developer would need maybe 2-3 days to build a working prototype
- Latency is real but manageable — expect 3-7 seconds from spoken words to suggestion appearing
- RingCentral's Live Transcription accuracy varies with accents and technical jargon — you'd want to feed it a CNC terminology glossary
- Claude has a context window limit, so long calls need smart summarization to avoid losing early context

How Hard Is This to Build?

Approach	Effort	Cost/mo
Prototype (internal web page)	2-3 dev days	~\$50-100
Polished Chrome extension	1-2 weeks	~\$100-200
Full integrated product	1-2 months	Scales with usage

If you don't have a developer, tools like **Replit** or **Cursor** have made this kind of project buildable by a motivated non-developer in a few weekends.

This is genuinely one of the highest-leverage things you could build for a technical support team. Want me to sketch out the actual code architecture or a prototype spec you could hand to a developer?

Ok, lets think bigger here. Here is what I am thinking: instead of building one piece or another, lets just do it right. Combine the best ideas from above. What would a system look like that could: log all ZenDesk tickets, RingCentral Calls, be able to use that info to diagnose current issues and issues our techs are on the phones with customers, and what about eventually creating an AI agent to answer ZD tickets or phone calls - if the customer wants to try it

This is a real product. Let me map the whole thing out properly.
This is a real product. Let me map the whole thing out properly.

☒ Connecting to visualize...

Click any box to go deeper on that piece. Here's the big picture narrative:

You're building this in three phases, not all at once.

Phase 1 — The brain (build this first). Get ZenDesk and RingCentral feeding into one central database. Every resolved ticket, every call transcript, every machine model and resolution gets stored, auto-categorized by Claude, and made queryable. This is the foundation everything else depends on. Timeline: 2-4 weeks with a developer, or 4-6 weeks with no-code tools.

Phase 2 — The co-pilot (highest ROI). Once the brain has data, you build the real-time sidebar your techs use on calls. They're still in control — Claude is just a quiet expert whispering in their ear. Suggested diagnosis, next question to ask, similar cases. This alone will cut average handle time and improve first-call resolution dramatically. A warrant flag also fires automatically if certain phrases are detected. Timeline: 4-8 weeks after Phase 1.

Phase 3 — The agent (when you're ready). Once the brain is rich enough — probably 6-12 months of data — you flip the switch on an AI agent that can handle ZenDesk tickets autonomously and, for customers who opt in, answer phone calls. It escalates to a human whenever confidence is low. This is not a replacement for your team; it's a way to handle the easy 40% automatically so your techs focus on the hard stuff.

The honest build cost: A competent developer could have Phase 1 running in a month. The full system is realistically a 3-6 month project. The good news: each phase delivers value on its own, so you're not waiting months for ROI.

Want me to go deep on any specific layer — the database schema, the co-pilot UI, or the AI agent architecture?

Why would this take so long with AI? Why would the brain take that long

Fair pushback. Let me be honest — with AI-assisted development, those timelines are way too conservative. Here's the reality:

What actually takes time (and what doesn't)

The things that *used* to take weeks are now hours:

- Writing the webhook receiver for ZenDesk? Claude Code generates that in minutes
- Setting up a Supabase database with the right schema? 30 minutes
- The Make.com automation? A few hours of clicking, no code required
- The Claude enrichment API call? Boilerplate — maybe an hour

What still takes real time:

- **RingCentral's Live Transcription API** has a real approval/provisioning process on their side. You have to request access, and they may take a few days to enable it on your account
- **Testing with real data** — you want to run a few weeks of actual tickets through it before you trust it
- **Iteration** — the first version of your Claude enrichment prompt will misclassify things. You'll tweak it based on what you see
- **Your own time** — someone on your team has to actually sit down and do this

Realistic timeline with AI tools

Phase	Realistic with AI
ZenDesk webhook → Supabase → Claude enrichment	1-3 days
RingCentral transcript logging (post-API access)	1-2 days of actual work
Waiting for RingCentral API provisioning	Up to a week (not your fault)
Co-pilot sidebar UI	2-5 days
Basic AI agent for ZenDesk replies	3-5 days

The whole thing could realistically be a working prototype in 2-3 weeks, with most of that time being waiting on RingCentral or your own schedule — not actual build time.

The old timelines assumed a developer writing everything from scratch, reviewing PRs, writing tests, going through sprints. With Claude Code or Cursor doing the heavy lifting, you're mostly in the role of *director* — describing what you want, reviewing output, testing it. That changes everything.

Do you have a developer on your team, or would you be building this yourself?

How would an AI agent handle ZenDesk tickets autonomously?

Great question — and this is actually the most *immediately* buildable piece of the whole system. Here's exactly how it works:

The Basic Loop

Every time a ZenDesk ticket comes in, instead of sitting in a queue waiting for a

human, it hits this flow automatically:

☐ Connecting to visualize...

Click any box for a deeper dive. Here's what makes each step work in practice:

Triage is Claude reading the ticket and extracting structure — machine model, issue category, customer history, whether it's in warranty, and how many similar tickets exist in your database. This takes about 2 seconds.

Confidence scoring is the key design decision. You define thresholds based on what you're comfortable with. A reasonable starting point:

- *High (auto-reply)*: Common issue, exact match in past tickets, clear resolution path — e.g. "my spindle speed display is frozen" has been solved 14 times the same way
- *Medium (draft for human)*: Recognizable issue but some ambiguity — Claude drafts the reply, tech reads it in 30 seconds and hits send or edits
- *Low (escalate)*: Novel issue, complex symptoms, or anything with safety implications — goes straight to a human, no AI reply attempted

The draft-for-human path is the real win in the short term. Your techs aren't replaced — they're just reviewing Claude's homework instead of writing from scratch. That alone can cut response time from hours to minutes.

Warranty flagging is automatic. Any ticket where Claude detects warranty-relevant language gets routed to escalation regardless of confidence score, with a flag on the ticket.

The feedback loop at the bottom is what makes it get smarter over time. Every resolved ticket — whether AI-handled or human-handled — feeds back into the knowledge base. In 6 months, your high-confidence pool is much larger than it is on day one.

The thing most people miss: you don't have to start with auto-reply at all. You can launch with *everything* going through the draft-for-human path, let your techs build trust in Claude's output over a few weeks, then gradually promote issue categories to auto-reply as confidence grows. That's actually the safer and smarter rollout.

Want me to show you what the actual prompt that drives all this would look like, or how you'd build the confidence scoring logic?

What would an AI phone agent for CNC support sound like?

Great question — and the best way to answer it is to just show you. Let me build you an interactive demo of what it would actually sound like.

ⓘ Claude's response was interrupted.

I am also open to moving away from RingCentral or ZenDesk or both they are "locked gardens" that is - if RingCentral or ZD does not want us to be doing this - and they would rather if we paid for their product. Are there other phone and ticket services that would allow for a more seamless AI experience. I would still like Humans involved like normal, until we roll out new feature to new feature

Really smart thinking. Both RingCentral and ZenDesk have AI ambitions of their own, which means they have financial incentive to gate the very APIs you need. Let me lay out the landscape honestly.

The core problem with "locked gardens"

RingCentral's Live Transcription API requires enterprise approval and they can revoke or reprice access. ZenDesk recently acquired its own AI layer (Fin, powered by Intercom, and their own "ZenDesk AI") — they want you buying *their* AI add-on, not routing around it with Claude.

Neither will block you outright, but they control the pipes.

Phone — RingCentral alternatives

The key metric here is **WebSocket access to raw audio or transcripts without approval gates**.

Twilio is the gold standard for AI-first phone. It's infrastructure, not a product — it gives you raw telephony you own completely. Their `<Stream>` verb sends live audio to any WebSocket you control. You pipe that to a transcription service (Deepgram is the best at CNC/technical jargon), then to Claude. No approval process, no gatekeeper. Your techs use a softphone of your choice on top. Downside: it's plumbing — you assemble it yourself, there's no pretty admin UI out of the box.

Bland.ai is newer and built specifically for AI phone agents. If Phase 3 (the AI answering calls) is on your roadmap, Bland is worth knowing about — it's designed from the ground up for exactly that use case and hands off to humans gracefully. Less mature than Twilio but moving fast.

Vapi.ai sits between the two — developer-friendly, built for AI voice agents, easier to stand up than raw Twilio, and fully open to Claude integration. Good for prototyping the co-pilot and the agent in the same platform.

Ticketing — ZenDesk alternatives

The key metric: **open API, no AI add-on upsell, webhook on every event**.

Linear is beloved by technical teams but geared toward software — probably not the right fit for customer support tickets.

Plain is interesting — it's a support tool built API-first with the explicit philosophy that *you* bring the AI. No AI upsell, no locked garden. Still relatively new but worth watching.

Freshdesk has a generous API and webhooks on everything. Less political resistance to external AI than ZenDesk. Free tier is usable. Not as polished but fully open.

Unthread routes support tickets through Slack — if your techs already live in Slack, this could simplify the whole stack. Every ticket is a Slack thread, fully API accessible.

The most interesting option: build a thin custom layer. With Supabase as your database and a simple web UI, you can have a ticket management system that is

entirely yours in a few days. No vendor risk, no upsell pressure, full API control. Your techs would lose some polish versus ZenDesk but gain total flexibility. This becomes more attractive as your AI layer gets more sophisticated — you can shape the UI around the AI rather than bolting AI onto someone else's UI.

The honest recommendation

	Stay	Switch
RingCentral	If API access gets approved and pricing is stable	Twilio if you want full control; Vapi if you want AI-native
ZenDesk	If team is trained on it and volume is low	Freshdesk for a safe swap; custom layer if you're building seriously

The most future-proof move is **Twilio + a lightweight custom ticket layer**, because you own every pipe. But it's also the most work to set up.

A practical middle path: **keep ZenDesk for now** (your team knows it, migration is friction), **swap RingCentral for Twilio or Vapi** (the phone side is where you need the most control for the co-pilot and agent features). You get 80% of the benefit with half the disruption.

Want me to sketch out what a Twilio + Vapi + custom ticket stack would look like, or compare Freshdesk vs a custom layer in more detail?

our ticket volume is actually fairly heavy. we get all sorts of stuff to our support email. Spam, solitations, complaints, quote requests, technicla questions, and so on and so forth. ZD has been critical to us and we are familiar with it - however, because our tickets are so varied, we still have humans routing tickets and manually replying. ZD also has the CSAT feature - which is important to us for capturing bad reviews before they become public

That context changes things significantly — and actually makes the case for staying on ZenDesk even stronger. Let me reframe.

ZenDesk is actually the right call for you

Heavy, varied volume is exactly what ZenDesk was built for. And the CSAT interception feature is genuinely hard to replicate — catching a frustrated customer before they post a Google review is worth real money. Don't touch that.

The question isn't "should we leave ZenDesk" — it's "**how do we make Claude do the work your humans are currently doing inside ZenDesk.**"

And that's very buildable.

What your humans are doing today that Claude can take over

You described the exact problem Claude is best at. Right now someone on your team is opening every email and making decisions like:

- Is this spam? → Delete/mark
- Is this a solicitation? → Ignore or route to purchasing
- Is this a complaint? → Flag, prioritize, assign
- Is this a quote request? → Route to sales
- Is this a technical question? → Assign to a tech

That's triage. Claude can do all of it in about 2 seconds per ticket, automatically, the moment it lands in ZenDesk. Your humans shift from *doing* the routing to *occasionally correcting* it.

☐ Connecting to visualize...

Click any box to go deeper. A few things worth calling out specifically for your situation:

On CSAT — this actually gets better with AI. Right now a complaint lands and sits in a queue until someone gets to it. With Claude, the moment a ticket is classified as a

in a queue until someone gets to it. With Claude, the moment a ticket is classified as a complaint it gets top priority, Claude drafts an empathetic response within seconds, and a human reviews before it goes out. The customer feels heard fast, which is the single biggest factor in whether they post a bad review or not. You're not just maintaining your CSAT interception — you're making it dramatically faster.

On spam and solicitations — this alone might justify the whole project. If even 20-30% of your ticket volume is noise, Claude silently handles that category entirely. Your team never sees it. That's real hours back every week starting day one.

On quote requests — this is an underrated opportunity. Claude can extract the machine model, configuration, and what they're asking for, pre-fill a quote template, and route it to sales with all the context already attached. Sales doesn't have to read the original email at all.

On the human-in-the-loop concern — nothing here removes your team. They go from routing everything manually to only reviewing what Claude flags as needing a human eye. Complex technical issues, complaints, anything with low confidence. The rest flows automatically. You dial up automation gradually as trust builds.

The ZenDesk API supports all of this natively — triggers, webhooks, tags, group assignments, internal notes, priority settings. You're not fighting the platform, you're just adding a Claude layer on top of what ZenDesk already exposes.

Want me to map out what the actual ZenDesk trigger and webhook setup looks like to make this run?

ok, so you recommend staying with ZD? What about RingCentral?

Yes, stay with ZenDesk. The CSAT feature alone is worth it, your team knows it, and the API does everything you need. Don't create migration pain for no gain.

RingCentral is the harder question

Here's the honest assessment:

The case for staying:

Your team already uses it, it works, and on the RX plan you have transcripts and some AI features built in. If Anthropic API access gets approved cleanly and the pricing doesn't punish you for piping data out, it's workable.

The case for leaving:

RingCentral is fundamentally a *communications company* that bolted AI on. They want to be your AI provider. Every time you route around their AI layer to use Claude, you're doing something they'd prefer you pay them for instead. That tension will show up as API friction, approval delays, pricing changes, or feature gates — not necessarily today, but eventually.

The Live Transcription API on RX is also not a guarantee — it's a provisioned feature they can change the terms on.

What I'd actually recommend

Short term — stay on RingCentral. Don't do two migrations at once. Get the ZenDesk AI layer built first, let your team adjust, build confidence in the system. RingCentral works fine for the co-pilot prototype.

Medium term — evaluate Twilio or Vapi once you're ready to build the phone co-pilot seriously. The difference in what you can build is significant:

	RingCentral RX	Twilio	Vapi
Live audio access	Approval required	Native, no gates	Native, built for AI
Transcript quality	Their model	You choose (Deepgram)	Deepgram built in
AI layer control	They want to own it	You own everything	Claude-native
Human handoff	Manual	Configurable	Built in
Cost at scale	Per seat + API fees	Pay per minute	Pay per minute
Setup complexity	Low	High	Medium
Phase 3 AI agent	Difficult	Buildable	Purpose-built

Vapi is worth a serious look for your specific roadmap. It's designed exactly for what you described — humans handling calls normally today, AI agent handling calls tomorrow, with a graceful handoff between them. You can run both in parallel during the transition. Your techs keep their existing workflow while the AI agent handles overflow or after-hours calls first.

The migration trigger to watch for

Stay on RingCentral until one of these happens:

- They deny or delay your Live Transcription API access
- They raise prices specifically on API/data export features
- You're ready to build Phase 3 (the AI phone agent) — at that point Vapi becomes clearly superior

Until then, the switching cost isn't worth it. Focus the engineering effort on ZenDesk first, get that running, then revisit the phone stack in 3-6 months with real data about what you need.

Does that framing make sense? Want to dive into what the ZenDesk build actually looks like in practice — the triggers, the Claude prompt, the routing logic?

Images hidden in
shared chats



this is last months ringcentral data to give some more context. You can see there are some issues i would like to go after

Now we're talking — this is really useful data. Let me read it carefully before commenting.

A few things jump out immediately:

The abandonment problem is severe

3,110 inbound calls last month. Only 1,550 answered. **1,560 abandoned** — that's exactly 50% of your callers hanging up before anyone picks up. That is a massive number. Half your inbound call volume is walking away, and you have no idea what they needed.

The Crossfire Pro XR Line queue is the worst offender — 538 abandonments out of 1,015 inbound calls (53% abandon rate) with a 9:11 average handle time. That queue alone is a fire.

Apollo Line is even more alarming proportionally — 136 abandonments out of 223 calls, a 61% abandon rate, and 11:30 average handle time. Nearly two-thirds of callers on that line give up.

The General Inquiry queue tells an interesting story

1,414 inbound, 491 holds, 722 abandonments — but a 2:48 average handle time. Calls are short when they get answered, but 491 people were put on hold and 722 gave up. This suggests a routing problem more than a capacity problem. Many of these are probably the spam/quote/solicitation mix you mentioned — calls that shouldn't be going to your techs at all.

The users tab shows something important

☒ Connecting to visualize...

A few things the data is telling you that go beyond just "add AI":

The 50% abandon rate is your biggest number. 1,560 people called last month and hung up before talking to anyone. You don't know if those were warranty claims, machine-down emergencies, or someone just trying to buy a part. That's lost revenue and lost customer satisfaction you can't even measure right now. An AI agent handling overflow — even just answering, gathering information, and saying "a tech will call you back within the hour" — immediately converts those into recoverable interactions.

Crossfire Pro XR and Apollo are capacity problems, not just AI problems. The 9-11

minute handle times combined with high abandonment means those queues are genuinely overwhelmed. The co-pilot helps here by cutting handle time — if Claude can surface the answer in 90 seconds instead of a tech spending 4 minutes searching, that's real capacity freed up.

TITAN is your benchmark. 29% abandon rate vs 50-61% everywhere else, with comparable handle time. Something about how that queue is staffed or structured is working. Before building anything, I'd find out what TITAN is doing differently — it might be the simplest win available.

Also notice what RingCentral is doing in that Users tab banner — they're advertising "AI Conversation Expert" to analyze your calls. That's exactly the tension I mentioned. They see your data, they want to sell you their AI layer on top of it. That's not necessarily bad, but it's worth knowing their AI product is designed to replace what you're trying to build with Claude.

The abandon rate alone makes the case for moving faster on this than you might have thought. What queue do you want to attack first?

Yes. So with this context, what path does this steer towards - sticking with ZD or going a different route? I am more open to changing phones than ZD. The only connection between the two is that missed call voicemail emails to to ZenDesk, but I assume there could be a better option.

What do you think

With this data, my thinking shifts a bit. Let me be direct.

ZenDesk — still yes, don't touch it

Nothing in this data changes that. Your ticket volume, CSAT needs, and team familiarity all point the same direction. Stay.

RingCentral — this data makes a stronger case for leaving

Here's why. Look at what you actually need:

- Handle a 50% abandon rate — meaning you need **AI to answer overflow calls**
- Reduce 9-11 minute handle times on Crossfire and Apollo — meaning you need a **real-time co-pilot**
- Capture those 1,560 abandoned callers' intent — meaning you need **every call logged and queryable**
- Eventually let AI answer calls autonomously for customers who opt in

RingCentral can technically do some of this. But you'd be fighting the platform the whole way — their API gates, their AI upsell banner already appearing in your dashboard, their incentive to keep you dependent on their intelligence layer.

Vapi changes the math completely for your situation.

Why Vapi fits your specific numbers

The 1,560 abandoned callers is the killer insight. Right now those people hit your queue, wait, give up, and disappear. With Vapi:

- Overflow calls that would have abandoned get answered by the AI agent instantly
- It gathers: name, machine model, nature of the issue, urgency
- It either resolves simple issues on the spot or creates a callback ticket
- That ticket goes directly into ZenDesk — better structured than a voicemail email

That one change alone could recover hundreds of customer interactions per month that you're currently losing entirely.

The co-pilot for your techs on Crossfire and Apollo — the queues with 9-11 minute handle times — is also cleaner on Vapi because you own the audio stream without asking anyone's permission.

The stack I'd recommend

The connection you mentioned — voicemail emails to ZenDesk

That's actually one of the weakest links in your current setup. A voicemail email is unstructured noise — someone has to listen to it, transcribe the relevant info, figure out what machine, what issue, what urgency, and then manually create a proper ticket.

With Vapi, that flow becomes: missed call → AI answers → gathers structured info → writes a clean ZenDesk ticket with machine model, issue category, customer name, and urgency pre-filled. A human just picks it up and acts. No listening to voicemails, no manual data entry.

That's not a worse version of what you have. It's genuinely better.

One honest caveat on Vapi

It's newer than RingCentral and less battle-tested at enterprise scale. Your team would also need to learn a new softphone setup. I wouldn't do a hard cutover — I'd run Vapi on one queue first (General Inquiry or overflow only), prove it out for 30 days, then expand. RingCentral stays live the whole time until you're confident.

Want me to map out what that 30-day pilot would look like?

Tell me more about Vapi. Give me an example of how it can be used effectively? How would my techs use it? (They like to answer the RingCentral App on their phones or computer currently). What admin controls will I have? What Metrics and Data will I be able to see and capture? Just provide some more detail

Great set of questions. Let me give you a real picture of Vapi — what it is, what it isn't, and how your team would actually experience it day to day.

What Vapi actually is

Vapi is not a phone system in the traditional sense. It's best understood as **AI voice infrastructure** — it handles the intelligence layer of a call (listening, understanding, responding, routing) and sits on top of regular phone carriers for the actual call delivery.

This distinction matters for your team: **Vapi doesn't replace the app your techs answer calls on.** It sits in front of it.

 Connecting to visualize...

How your techs would actually use it

This is the part most people get wrong about Vapi — they assume it forces a new workflow on the team. It mostly doesn't.

Phase 1 (where you start): Vapi only handles overflow and after-hours. All 6 of your queues work exactly as they do today. When a call would have gone abandoned — because all techs are busy — Vapi picks it up instead. Your techs don't change anything. They still answer on the RingCentral app on their phone or computer exactly as today.

Phase 2 (co-pilot live): Vapi now answers every inbound call first — but only for about 30-45 seconds. It asks the caller their name, machine model, and brief issue description. Then it transfers to the first available tech. The tech's screen (or a simple browser tab) shows the gathered info before they even say hello. They walk into every call already knowing the machine and the problem. The RingCentral app still rings. Nothing else changes for them.

Phase 3 (AI agent option-in): For customers who choose it, Vapi handles the full call. Your techs don't see these unless Vapi decides to escalate. They appear in ZenDesk as resolved tickets.

The "whisper" feature is worth calling out specifically — right before Vapi connects the customer to a tech, it plays a 5-10 second audio clip *only the tech hears*: "Incoming call — John at Acme Manufacturing, Crossfire Pro XR, Z-axis fault, he's called about this twice before." The tech picks up already oriented. That alone shaves 2-3 minutes

this twice before. The tech picks up already oriented. That alone shaves 2-3 minutes off every call.

Admin controls

Vapi has a dashboard where you control:

Voice and persona — you choose the AI voice (there are dozens, including very natural-sounding ones), give it a name, set its tone. For CNC support you'd want something professional and calm, not cutesy. You write the personality in plain English: *"You are a support intake agent for [Company]. Be professional, efficient, and empathetic. Never guess at technical answers — always transfer to a human tech for diagnosis."*

Call routing rules — which queues the AI handles, in what order, what triggers a transfer vs a resolution. You set these per queue, so Crossfire Pro XR can have different rules than General Inquiry.

Escalation triggers — words or phrases that immediately bump to a human, no AI attempt. You'd set things like: "machine down," "safety issue," "warranty claim," "I want to speak to a person." Customer always wins.

Knowledge base connection — you point Vapi at your Supabase database and it draws on past resolved cases during the call. You control what it has access to.

Business hours and overflow rules — after-hours behavior, holiday schedules, what happens when all techs are busy at 2pm on a Tuesday.

Per-queue configuration — Apollo Line can have completely different AI behavior than TITAN. You tune each one independently.

Metrics and data you'd capture

This is where Vapi genuinely beats RingCentral for your use case, because every call generates structured data, not just call logs.

Per call you get: full transcript, call duration, AI vs human resolution, transfer reason, machine model mentioned, issue category, sentiment score, whether the caller was frustrated, how long they waited before transfer, and the ZenDesk ticket that was created

created.

Across all calls you can track:

- AI resolution rate by queue and by issue type
- Transfer rate and transfer reasons — tells you what the AI isn't good enough to handle yet
- Abandon rate (this should drop fast once AI handles overflow)
- Average time-to-answer — no more 9 minute waits
- First-call resolution rate
- Sentiment trends by machine model — if Crossfire Pro XR callers are consistently frustrated, that's a product signal
- After-hours call volume and what those callers needed — you're currently losing all of this

The thing that doesn't exist in your RingCentral data today but would exist in Vapi: **you'd know what every single one of those 1,560 abandoned callers actually needed.** Because Vapi would have answered them, gathered their info, and logged it — even if they hung up before reaching a tech.

The honest limitations

Vapi is newer and the dashboard is less polished than RingCentral's. Your techs will notice the difference in the admin UI. It also doesn't have a native mobile app as slick as RingCentral's — though your techs can still receive transferred calls on any device since Vapi connects to standard phone numbers. And AI voice, while very good now, still occasionally sounds slightly unnatural on complex back-and-forth. That's why the human handoff is always one word away.

The 30-day pilot on one queue approach I mentioned earlier is the right way in — prove it on General Inquiry overflow first, where the bar is lowest, then expand with confidence.

Want me to sketch out what that pilot plan would look like week by week?

ok, our customer base is like very rural people. I think some may negatively react if an AI agent is the first thing they hear. Is it possible to

negatively react if an AI agent is the first thing they hear. Is it possible to add the AI Agent to the end of a call routing? I.e. its about to be missed or abandoned.

I could be wrong here, I am open to better ideas. I just dont want customers negatively reacting to AI

You're not wrong — that instinct is good. Rural, trade-based customers who work with heavy machinery tend to value straight talk and real people. An AI voice as the *first* thing they hear could absolutely rub some of them the wrong way.

But here's the thing — **your framing of "end of the queue" is exactly right, and it's actually the best practice even for customers who don't have that sensitivity.**

The "last resort catch" model

Instead of AI-first, you build it as a safety net:

- Call comes in
- Routes to your techs normally, exactly as today
- Rings through your queue for 60-90 seconds
- If nobody picks up — the call is *about* to be abandoned — Vapi catches it

At that point the customer has already waited. They're about to hang up and get nothing. An AI catching them is strictly better than silence. Most people, rural or not, prefer "someone" answering over a dead queue.

But the framing of *how* it answers matters enormously

This is where most companies get it wrong. They lead with "Hi, I'm an AI assistant..." and immediately put people off.

The better approach is **task-first, technology-second**:

"Hi, thanks for holding — all of our technicians are currently with other customers. I can take down your information and have someone call you back within the hour, or I can try to help with some common issues right now. Which would you prefer?"

No mention of AI. No robot disclaimer upfront. Just a helpful voice offering two useful options. If they ask "am I talking to a robot?" — it answers honestly. But most people won't ask if the experience feels helpful.

The callback offer is key for your customer base specifically. A rural customer who's been waiting doesn't want to talk to AI — but they absolutely want a callback from a real tech. Vapi capturing their name, number, machine, and issue and promising a callback is something they'll appreciate regardless of what's delivering it.

A middle path worth considering

☐ Connecting to visualize...

One more thought on your customer base specifically

You might be underestimating how much your rural customers will *appreciate* the callback option. These are people who are often on a job site, can't be on hold, and may have called during a window when they had signal. What they hate isn't AI — **what they hate is being ignored.**

Right now 1,560 of them last month got exactly that — silence and a missed call. A voice that says "we've got your info and a tech will call you back within the hour" — even if that voice is AI — is a dramatically better experience than what they're getting today.

You could also consider **not branding it as AI at all** for now. Just a helpful overflow system. As your customers get comfortable with it and you see the CSAT scores, you can decide how much to lean into it. You're not obligated to announce the technology — you're just obligated to serve your customers well.

The rollout I'd suggest: start with after-hours only. Zero customers hear it during business hours. You capture after-hours abandoned calls, build the Zendesk tickets, your techs call back in the morning. No customer reaction risk, immediate value, and you start learning what those after-hours callers actually need. Expand from there as you gain confidence.

I would also like better flexibility for a phone system that calls back customers and forces the tech to answer the phone. Would Vapi have that possibility

Yes — and this is actually one of Vapi's stronger features. It's called **outbound calling** and it can be fully automated and forced.

Here's what that would look like for your team:

The "forced answer" callback flow

When a customer hits the AI overflow and requests a callback, or when a ZenDesk ticket gets flagged as high priority, the system can:

1. **Automatically call the customer back** at a scheduled time or immediately when a tech becomes available
2. **Simultaneously call the tech** — not a notification, an actual ringing phone call they have to pick up
3. **Bridge them together** only when both parties answer
4. If the tech doesn't answer within a set time, it calls the **next available tech** automatically
5. Logs everything — who called, who answered, how long it took, outcome

The key word you used is "forces" — and that's exactly right. A Slack notification gets ignored. An email gets ignored. A phone that won't stop ringing until you pick it up doesn't get ignored.

A few specific scenarios this solves

The high-priority callback queue. Machine-down customer, been waiting 2 hours, ticket is red in ZenDesk. System calls tech's cell directly. Tech hears: *"You have a priority callback — customer John at Acme Manufacturing, Crossfire Pro XR machine down. Press 1 to connect."* Tech presses 1, customer is already on the line. Zero manual coordination.

The after-hours emergency. Customer calls at 7pm, AI takes their info, flags it urgent. System holds until 8am, then auto-dials the on-call tech. No one has to monitor a queue overnight.

The missed callback promise. Vapi told a customer a tech would call within the hour. 55 minutes pass, no tech has called. System automatically escalates — calls a second tech, or a supervisor, before the promise is broken.

Round-robin outbound. Multiple callbacks queued up at 8am Monday. System works through them sequentially, calling each tech in rotation so no one person gets buried.

How techs would experience this

This is worth being honest about — some techs will initially resist a system that *forces* them to answer. The framing matters.

The way to position it internally: *"The system is doing the dialing for you. You don't have to find the ticket, look up the number, dial manually, and remember to log it. Your phone rings, you pick up, the customer is already there. That's it."*

Most techs, once they experience it, prefer it. The manual callback process — find ticket, read notes, dial, leave voicemail, update ticket, repeat — is genuinely tedious. This collapses it to one step.

What you'd control as admin

- **Response time SLAs per queue** — Crossfire Pro XR callbacks within 30 minutes, General Inquiry within 2 hours
 - **Escalation chain** — who gets called first, second, third if no answer
 - **Hours of operation** — don't force-call techs at midnight unless they're on-call
 - **On-call scheduling** — rotate which tech is on the escalation list by day or week
 - **Priority tiers** — machine-down calls jump the queue ahead of general questions
 - **Voicemail behavior** — if customer doesn't answer the callback, leave a message or try again in 30 minutes
-

The ZenDesk connection here is valuable

Every one of these outbound calls writes back to the ZenDesk ticket automatically — time of call, which tech answered, duration, outcome. Your supervisors see the full chain: customer called at 2pm, AI took message at 2:03, tech called back at 2:41, resolved at 2:54. No manual logging, full accountability.

That's actually something you can't easily do with RingCentral today — the inbound and callback data lives in separate places and someone has to stitch it together manually.

This callback forcing mechanism, combined with the overflow catch we talked about, is honestly the most immediately impactful thing you could build. It directly attacks your two biggest problems from the data — the 1,560 abandoned calls, and the fact that you currently have no visibility into what happened after those customers hung up.

Want me to sketch out the full callback workflow end to end, including what the tech and customer each hear?

Tell me more about vapid. Is it an app that works in RingCentral? I'm still not understanding what it is

Good question — and I want to make sure I've been clear here. First, a small correction: it's **Vapi** (not Vapid) — easy to mishear.

But more importantly — I've been recommending Vapi confidently in this conversation and I want to be honest with you: **my knowledge of Vapi's current features, pricing, and capabilities may be outdated**. Vapi is a fast-moving startup and things may have changed since my knowledge cutoff. I don't want you making a phone system decision based on details I can't fully verify.

Let me tell you what I know with confidence, and flag what you should verify directly with them.

What Vapi definitely is

Vapi is **not an app inside RingCentral**. It's a completely separate platform. Think of it less like a phone app and more like a programmable phone brain that you connect to your existing phone numbers.

The simplest analogy:

RingCentral is a complete phone office — handsets, voicemail, call routing, the whole thing. Vapi is more like a smart receptionist service you hire that answers your phone lines and transfers to your real office.

It's developer-oriented infrastructure, not a plug-and-play consumer app. That's important for your team to understand — there's setup involved, and it likely requires a developer or technical resource to configure properly.

What you should actually do right now

Honestly, rather than me going deeper on Vapi specifics that I can't fully guarantee are current, I'd point you in two directions:

- 1. **Vapi's own documentation** at `vapi.ai` — they have demos, pricing, and use case examples that will be more accurate than anything I tell you today.
- 2. **Consider these alternatives in the same category** that are worth comparing:

Platform	What it is	Best for
Vapi	AI voice infrastructure, developer-first	Custom AI agent builds
Bland.ai	Similar to Vapi, slightly more turnkey	AI phone agents
Twilio	Raw telephony infrastructure	Maximum control, more dev work
Aircall	Modern cloud phone system, strong API	Teams wanting RingCentral replacement with better integrations
OpenPhone	Simple modern business phone	Smaller teams, easy setup

The honest reframe

I've been treating this as a binary — RingCentral vs Vapi. But given what you've told me about your team (they like the RingCentral app on their phones and computers, they're not highly technical), there's actually a question worth asking first:

Do you want to build a custom AI phone system, or buy one that's closer to ready?

Because Vapi and Bland are really for teams that want to *build* — a developer configures the AI behavior, connects the APIs, writes the logic. It's powerful but it's a project.

If you want something closer to plug-and-play with strong AI features, **Dialpad** might be worth a serious look. It has:

- Built-in real-time transcription on every call
- AI call summaries automatically
- Coaching features for techs
- A proper mobile and desktop app your team would recognize
- ZenDesk integration out of the box
- Callback and voicemail features

It won't give you the fully custom AI agent we've been designing — but it gets you 60% of the way there without a developer, and your techs would barely notice the transition from RingCentral.

My recommendation given everything

Rather than me steering you toward a specific platform based on potentially stale information, here's what I'd actually do in your position:

Take the list of capabilities you need — and we've mapped them out well in this conversation — and have short demo calls with 2-3 vendors. Your requirements are clear:

- Real-time transcription on every call
- Overflow AI catch before abandonment
- Forced outbound callback to techs
- Clean ZenDesk integration
- Mobile and desktop app your techs recognize
- Ability to grow into a fuller AI agent over time

Any serious vendor in this space should be able to tell you clearly whether they support each of those. The ones that hesitate or oversell are the ones to avoid.

Want me to help you build out that requirements list as a formal vendor evaluation scorecard you could take into those conversations?

This is just an exploratory conversation. How does Vapid or Bland interact with RingCentral? Does it require the same api pipes issue?

Ha — Vapi, not Vapid! Though I understand why it keeps coming out that way.

And this is the right question to ask. Let me give you the straight answer.

They don't interact with RingCentral at all

Vapi and Bland are not add-ons to RingCentral. They don't plug into it, integrate with it, or sit on top of it. They **replace** it entirely for the lines they handle.

The way it actually works:

You get a phone number from Vapi or Bland — just like you'd get a number from any phone company. Customers call that number. Vapi/Bland owns that line completely. RingCentral is not involved at all.

So there's no API pipe issue with RingCentral because RingCentral isn't in the picture on those lines.

The practical question this raises

If you moved to Vapi or Bland, your options would be:

Option A — Full replacement. Port your existing phone numbers over to Vapi/Bland. RingCentral goes away entirely. Your techs need a new way to receive transferred calls — could be any softphone app, their mobile, or a desk phone.

Option B — Parallel running. Keep RingCentral for your techs to answer calls on. Vapi/Bland handles the AI layer — overflow, intake, callbacks — and transfers to your RingCentral numbers when a human is needed. Both systems run alongside each other. This is actually how most companies start.

Option B is cleaner for your situation because your techs keep using exactly what they use today. Vapi just sits in front of RingCentral as the first layer, handles what it handles, and passes the rest through.

So the API tension we talked about earlier

That was specifically about trying to use RingCentral's *own* live transcription API to pipe data out to Claude — which is where they have incentive to gate you.

With Vapi or Bland running their own lines, that problem doesn't exist. You own the data on those lines completely. No permission needed from RingCentral because RingCentral isn't touching those calls.

Bottom line for this exploratory conversation

Think of it this way:

- **RingCentral** = the phone company and the app your techs use
- **Vapi/Bland** = a smarter phone company that also happens to be an AI

They operate at the same level. You're not adding one on top of the other — you're choosing between them, or running them side by side on different lines during a transition.

Does that clarify the relationship? Happy to keep exploring before you go anywhere near an actual decision.

Does vapi have admin controls and routing to techs like my current set up does? Would it allow the logging of call transcripts that I could then build a database?

Great questions — and these get to the heart of whether Vapi is actually a viable RingCentral replacement for your team day-to-day.

Here's my honest answer, split into what I'm confident about and what you should verify:

What I'm confident about

Transcripts and database logging — yes, absolutely. This is core to what Vapi is built for. Every call generates a full transcript, and Vapi has webhooks that fire at the end of every call sending you the complete data — transcript, duration, outcome, caller info, everything. That's exactly what you'd pipe into Supabase to build the knowledge base we've been talking about. This is not a workaround or an edge feature — it's the primary reason people choose Vapi over traditional phone systems.

Call routing to humans — yes, this works. Vapi can transfer calls to any phone number. So "transfer to tech queue" just means calling your techs' numbers. Simple.

Where I'd slow down

Admin controls and queue management — this is where I want to be careful. The queue management, presence indicators, call waiting, hold music, team dashboards, extension dialing, and all the day-to-day operational controls you're used to in RingCentral? Vapi was built by developers for developers. Its "admin panel" is more of a configuration dashboard than an operations center.

Think of it this way:

Feature	RingCentral	Vapi
---------	-------------	------

Call queues with hold music	Built in, GUI	Configurable, technical setup
Agent presence / availability	Built in	You build it
Team extensions	Built in	Not native
Live call monitoring	Built in	Limited
Admin dashboard for non-technical managers	Strong	Weak
Transcript logging	Limited/gated	Excellent, native
AI call handling	Their AI product	Fully custom with Claude
Outbound callback automation	Basic	Strong

What this tells me about your situation

You have a real operational phone system with 13 agents, 6 queues, hold routing, and a team that depends on it daily. Vapi as a *full replacement* for that would require building a lot of the operational layer yourself — queue management, availability, monitoring — that RingCentral just gives you out of the box.

That's not a dealbreaker but it's real work, and it means you'd need a developer not just for the AI features but to rebuild the basic phone operations infrastructure too.

The architecture that actually makes sense for you

Given everything you've told me, I think the right mental model is this:

RingCentral — stays as your operational phone system

↓

Vapi — handles AI overflow and callback layer only

↓

Supabase — captures transcripts from both

↓

Claude — the brain that uses all of it

You don't replace RingCentral with Vapi. You add Vapi specifically for the jobs RingCentral can't do — AI overflow catch, forced callback, after-hours handling, transcript logging. RingCentral continues doing what it does well — giving your 13 agents a proper phone system with queues, presence, and monitoring they already know.

The API tension I mentioned earlier? For this specific use case — Vapi handling overflow lines, not your main lines — it's not an issue. Vapi owns those calls completely. RingCentral owns your main tech queues. They don't need to talk to each other except for a simple call transfer.

The one thing to verify directly with Vapi

Ask them specifically: *"We want to use you for AI overflow handling and outbound callbacks only, while keeping our existing RingCentral setup for our human agents. Is that a supported use case and how do other customers do it?"*

That question will tell you quickly whether they've solved this for teams like yours or whether you'd be pioneering it yourself.

Does this framing help clarify where Vapi fits — and more importantly where it doesn't?

If I went full Vapi, would I be able to get a real time transcript with the Claude prompt and Screens for a tech on the phone with a customer

Yes — that's actually Vapi's sweet spot. A full Vapi setup is what makes the real-time co-pilot screen genuinely possible, because you own the entire audio stream without asking anyone's permission.

Let me show you what that tech screen would actually look like in practice.

Viewed a file, created a file, read a file



Log in to see interactive visuals

That's a live mockup of what your techs would see on their screen during a call. The timer is actually counting up. Here's what each piece does in real life:

Top bar — caller name, company, which machine they own, and how many times they've called this month. Your tech knows Randy before they say a word.

Tag bar — auto-populated from the first 30 seconds of the call. Vapi's transcription picks up "Crossfire Pro XR" and "E-47" from what the customer says, Claude classifies it, and the tags appear instantly. Machine down gets flagged red automatically.

Live transcript — scrolls in real time as the conversation happens. Both sides. The tech can glance down and see exactly what was said without losing their train of thought.

Claude's diagnoses panel — this is the co-pilot brain. As the conversation develops, Claude is reading the transcript, cross-referencing your knowledge base of past tickets, and surfacing the most likely causes ranked by confidence. By the time Randy mentions the storm, the 87% servo surge diagnosis is already on screen.

Call history — every previous call Randy has made, what it was about, how long it took, how it resolved. The tech can see immediately this is a customer with recurring issues, which changes how they handle the conversation.

Ask Claude — quick-fire questions the tech can tap mid-call. These open a separate window so they don't lose their place. "Does a power surge void warranty?" is a question a tech might spend 3 minutes hunting for in a manual — here it's one tap.

To answer your original question directly: yes, full Vapi gives you all of this cleanly because you own the audio stream entirely. With RingCentral you'd be working around API gates to get the same result. That's the real practical difference between the two for your use case.

Is it possible for me to get the last like 300 call transcripts as a ring central admin

Yes — and this is actually a great first move before you commit to anything.

As a RingCentral admin on the RX plan you have access to the **Analytics Portal** and the **RingCentral API**, and transcripts are stored there. Here's the honest breakdown:

What you can get right now as an admin

Through the RingCentral Admin Portal (no code):

- Go to **Analytics → Conversation Intelligence**
- You should see call recordings and AI-generated transcripts there if your RX plan has the Conversation Intelligence feature enabled
- You can export individual transcripts manually
- Bulk export is limited through the UI — you might be able to download a CSV of call summaries but full transcripts in bulk usually requires the API

Through the RingCentral API (requires a developer or someone technical):

- The `/call-log` endpoint gives you call records
 - The `/ai/insights` endpoint on RX gives you transcripts and summaries
 - 300 calls is completely feasible — you could pull them in one afternoon
-

The honest caveat

I'd recommend you log into your admin portal **today** and just look around before assuming anything. Specifically check:

- **Analytics → Conversation Intelligence** — are transcripts already there?
- **Analytics → Performance Reports → Calls tab** — can you export call logs?
- Whether your specific RX tier includes Conversation Intelligence or if it's a separate add-on

RingCentral's feature availability varies by exact plan tier and sometimes by what was enabled during your contract. Some RX customers have full transcripts sitting there they've never looked at. Others find it's behind an additional paywall.

What to do with 300 transcripts once you have them

This is actually the perfect way to start this whole project without building anything yet. If you can get those transcripts exported — even as a messy batch of text files or a CSV — you can drop them into a Claude Project right now and immediately start asking questions like:

- "What are the 10 most common issues customers call about?"
- "Which machine models generate the most support calls?"
- "What percentage of calls mention warranty?"
- "What issues are taking the longest to resolve?"
- "Are there any patterns in calls that ended with a frustrated customer?"

That analysis would tell you in a single afternoon what your support operation actually looks like — and give you the data to make a much more informed decision about where to invest in AI first.

It would also seed the beginning of your knowledge base. Those 300 transcripts are your first training data.

Practical suggestion

Before anything else in this conversation, go check your admin portal and see what transcript data is already sitting there. It might be more accessible than you think — and it gives you something real to work with immediately rather than waiting for a full system build.

Want me to help you figure out what to do with them once you have them?

Could I add like order history and other data into the Claude brain. I.e. we can see order numbers, dates, what he ordered, etc

Absolutely — and this is where the system goes from a support tool to something

genuinely powerful. Customer context during a call changes everything.

Think about what that Randy Hoffmann call would have looked like if your tech could also see:

"Randy bought his Crossfire Pro XR 14 months ago. He's ordered \$4,200 in consumables since. His servo driver is still under warranty. He last ordered Z-axis components 6 weeks ago."

That's a completely different conversation. The tech already knows the machine age, warranty status, and whether this is a loyal customer worth going the extra mile for — before Randy finishes his first sentence.

What data you can pipe in

Anything that lives in a database or has an API can feed the brain. For a CNC company the obvious ones are:

Order history — order numbers, dates, what was purchased, dollar amounts, how often they buy consumables. This tells you customer value at a glance.

Machine registration — which machine, when it was purchased, serial number, what warranty tier they're on. Eliminates the "can you tell me your model number" back-and-forth entirely.

Parts and service history — what parts have been replaced, any prior service calls or technician visits, what firmware version they're running.

Open quotes — if sales has a quote sitting with them, your support tech can see it. Avoids awkward moments where a customer references a conversation support knew nothing about.

Warranty status — calculated automatically from purchase date and machine type. Flags on screen the moment the call connects if they're in warranty, out of warranty, or in a grey zone.

Account notes — anything your sales team has flagged. "Prefers email follow-up." "Decision maker is the shop foreman not the owner." "Had a bad experience in 2023, handle carefully."

How it actually connects

The architecture is simpler than it sounds. Your order data lives somewhere today — whether that's an ERP system, a Shopify store, QuickBooks, a custom database, or even a spreadsheet. Almost all of these have either an API or an export capability.

You pull that data into Supabase alongside your call transcripts and ZenDesk tickets. Claude then has access to the full picture for any customer.

The moment a call comes in and Vapi identifies the caller's phone number, the system does a lookup:

Phone number → Customer record → Order history

- Machine registration
- Warranty status
- ZenDesk ticket history
- Call transcript history

All of that loads on the tech's screen before they even pick up. The "whisper" we talked about earlier gets much richer too — instead of just "Crossfire Pro XR, Z-axis fault," your tech hears "Randy Hoffmann, loyal customer, \$4,200 in orders, machine under warranty, third call this month."

What your system currently uses for orders

This is the key question I'd want to know before going further. Where does your order data live today? The answer changes how complex the integration is:

Order system	Integration difficulty
Shopify	Easy — strong API, well documented
QuickBooks / NetSuite	Medium — API exists, needs mapping
Custom ERP	Depends — varies widely
Spreadsheets / CSV	Easy — just load into Supabase directly
Multiple systems	Medium — needs a sync layer

The good news is you don't need a perfect real-time sync to start. Even a nightly export of your order data into Supabase gives your techs yesterday's information — which for support purposes is almost always good enough. Real-time sync is a phase 2 refinement, not a requirement on day one.

The compounding effect over time

This is the part worth pausing on. Every call that runs through this system adds to the knowledge base. After six months you don't just have order history and call transcripts — you have a complete picture of every customer's relationship with your machines.

Claude starts to recognize patterns you'd never catch manually:

- Customers who call about Z-axis faults within 18 months of purchase on a specific model — maybe there's a batch issue
- Customers who buy consumables heavily but haven't called support — they're probably happy and worth a proactive check-in
- Customers who've called three times in 30 days — they're at risk of a bad review or a return, flag them for a manager call
- Machine models that generate disproportionate support calls relative to units sold — product feedback your engineering team needs

That last category alone — surfacing product defects from support patterns — is something most companies only figure out after a flood of complaints. Your system would catch it in week two.

What does your order data situation look like today — do you know what system it lives in?

Ok, good discussion. Lets go back to the basics beginning and core of any system we build: the database. Tell me how we could begin this process. And if this grows into something important to the business, we will want the database in a secure place (not like somones personal google account)

Great instinct — the database is exactly the right place to start. Everything else we've talked about is just plumbing that feeds into it or draws from it. Get this right and the rest gets easier.

First, what this database actually needs to do

Before picking a technology, it's worth being clear on what you're asking it to handle:

- Store call transcripts and ZenDesk tickets permanently
- Be queryable by Claude in real time during a call
- Accept data from multiple sources — RingCentral/Vapi, ZenDesk, your order system
- Scale as volume grows without you thinking about it
- Be secure enough that customer data, warranty info, and business intelligence isn't sitting in someone's personal Google account
- Be accessible to a developer building the co-pilot and agent features later

The options, honestly ranked for your situation

Supabase is what I'd recommend for you. Here's why it fits specifically:

It's built on PostgreSQL — the most battle-tested open source database in the world, used by companies like Apple, Instagram, and Spotify. Supabase wraps it in a clean dashboard, handles authentication, gives you a real-time API automatically, and has a generous free tier to start. Most importantly, you own the data entirely — it's not locked inside RingCentral or ZenDesk's walls.

The other options worth knowing:

	Supabase	PlanetScale	MongoDB Atlas	AWS RDS	Google Cloud SQL
Best for	Your use case	High-scale apps	Flexible data	Enterprise	Enterprise
Setup time	20 minutes	30 minutes	20 minutes	Hours	Hours

Technical skill needed	Low-medium	Medium	Low-medium	High	High
Cost to start	Free	Free tier	Free tier	~\$50/mo	~\$50/mo
Scales to enterprise	Yes	Yes	Yes	Yes	Yes
You own the data	Yes	Yes	Yes	Yes	Yes
Dashboard for non-devs	Good	Okay	Good	Poor	Poor

AWS and Google Cloud are where large enterprises live — they're powerful but overkill for where you're starting, and they require more DevOps knowledge to manage. You can always migrate later if you outgrow Supabase, but most companies never need to.

On your security concern — this is the right thing to be thinking about

"Not in someone's personal Google account" is exactly the right instinct and here's how you protect against it properly:

Organization account, not personal. Supabase lets you create an organization with its own billing, ownership, and access controls. The database belongs to the company, not a person. If that person leaves, nothing walks out the door with them.

Role-based access. You control who can see what. A developer gets database access. A tech gets read-only access to customer records. A manager gets reporting access. Nobody gets more than they need.

Encryption. Supabase encrypts data at rest and in transit by default. Customer data, call transcripts, order history — all encrypted.

Backups. Automatic daily backups on paid plans. You're not one accidental deletion away from losing everything.

Compliance. If you ever need SOC 2 compliance — which matters if enterprise customers ask about your data practices — Supabase supports it. You're not painting yourself into a corner.

What the database actually looks like

Here's the schema I'd start with — the tables and what lives in them:

Created a file, read a file

That's your starting schema — every table your system will eventually need, with a clear build order so you're not trying to do everything at once.

A few things worth calling out:

The `ai_interactions` table is more important than it looks. Every time Claude classifies a ticket, suggests a diagnosis, or drafts a reply — that action gets logged with what it did and whether a human approved it or corrected it. Over time this tells you exactly how accurate Claude is, where it's improving, and where it needs better training. It's also your audit trail if a customer ever questions how something was handled.

The `knowledge_articles` table is what gets smarter over time. Every resolved call and ticket can generate or update an article. E-47 fault on Crossfire Pro XR after a power surge — that becomes a knowledge article with a resolution rate tracked every time it's used. After six months you have a living, self-improving repair manual built entirely from your own support history.

Start with just four tables. Customers, machines, tickets, calls. That's it. You can seed it immediately with your 300 RingCentral transcripts and your ZenDesk export. Orders, knowledge articles, and the rest come later when the foundation is solid.

How to actually get started this week

Three concrete steps that don't require a developer:

Step 1 — Create the Supabase organization account. Go to supabase.com, create an account under a company email, set up an organization (not a personal account). This takes 15 minutes and costs nothing. The database lives in the company's name from day one.

Step 2 — Export what you have. Pull your ZenDesk tickets as CSV and whatever RingCentral transcripts you can get from the admin portal. Don't worry about

formatting — just get the raw data out.

Step 3 — Drop it in Claude right now. Before any database is built, upload those exports to a Claude Project. Ask Claude to analyze them. You'll learn more about

your support operation in one afternoon than you have in months — and it directly informs what the database needs to prioritize.

The actual database build — creating the tables, setting up the pipelines, connecting ZenDesk webhooks — that's when you bring in a developer. But steps 1-3 you can do yourself, this week, with no technical help at all.

Want me to create a one-page brief you could hand to a developer that captures everything we've discussed — the schema, the integrations, the phases, the goals?

I am thinking multiple facets here. Could this database be used to create a chat bot? But I don't want the chat bot to tell customer information if asked. Like I don't want someone to ask the chat bot how many customers we have etc. Would I need to make a separate database for that? What do you think

Really smart to think about this now — it's much easier to design security in from the start than bolt it on later.

The good news: **you don't need a separate database.** You need separate *access layers* on top of the same database. This is a well-solved problem in software architecture.

The core concept: the database doesn't talk to anyone directly

Nothing ever queries your database directly — not the chatbot, not Claude, not your techs. Everything goes through a controlled layer that decides what each "user" is allowed to see.

Think of it like a restaurant. One kitchen, but the waiter only brings you what you ordered — they don't hand you the whole menu inventory, the supplier invoices, or the other tables' orders. Same data source, controlled views.

Your Supabase database (everything)



Access control layer



Customer

Tech

Admin

chatbot

co-pilot

dashboard

(public)

(internal)

(you)

What each "view" of the database sees

Customer chatbot — the most restricted view by far. It can only see:

- Knowledge articles and fault code resolutions
- The specific customer's own order history and machine info — and only if they're authenticated
- General product information
- It sees **zero** aggregate business data. Zero other customers. Zero ticket counts. Zero revenue numbers.

Tech co-pilot — sees more, but still scoped:

- Full customer record for whoever is on the call
- That customer's order history and machine history
- All call transcripts and tickets for that customer
- Knowledge base articles
- Still can't see aggregate business metrics or other customers' private data

Admin dashboard — that's you and your managers:

- Everything. Aggregate metrics, team performance, trends, all customers, revenue patterns, the works.

How Supabase enforces this technically

Supabase has a feature called **Row Level Security (RLS)** — it's built in and it's exactly what you need. You write simple rules like:

- Chatbot role: can only read `knowledge_articles` and `orders` where `customer_id = authenticated user`
- Tech role: can read all tables but only for the current active call's customer
- Admin role: full read access, no restrictions

The chatbot literally **cannot** return how many customers you have — not because Claude refuses, but because the database query returns nothing. The data is invisible at the infrastructure level, not just filtered by Claude's judgment.

On the chatbot specifically

This is a great use of the same database. A customer-facing chatbot sitting on your website could:

- Help a customer troubleshoot their specific machine
- Look up their own order status and history
- Check their warranty status
- Answer common questions about their model
- Create a support ticket if they can't self-resolve
- Escalate to a human tech if needed

What it can never do, by design:

- See any other customer's data
 - Return aggregate numbers about your business
 - Access pricing strategy or internal notes
 - See your team performance data
-

The one thing worth separating

There's one piece I would actually keep in a separate table — or at minimum tag clearly — and that's your **internal business intelligence data**. Things like:

- Aggregate revenue numbers
- Team performance metrics

- Product defect pattern analysis
- Competitor notes or pricing strategy

Not because the chatbot could accidentally access it — RLS prevents that — but because it's cleaner to have a clear architectural line between "customer-facing data" and "internal business intelligence." Makes audits easier, makes compliance easier, and makes it obvious to any developer what's public-safe vs business-sensitive.

Bottom line

One database, multiple access layers. Supabase's built-in Row Level Security handles all of it. The chatbot is not only possible — it's one of the highest-value things you can build on top of this foundation, because every customer interaction it handles is one fewer call or ticket your team has to deal with.

And here's the compounding effect: every chatbot conversation gets logged back into the database too. So the chatbot is simultaneously serving customers *and* feeding the knowledge base that makes Claude smarter for your techs on calls.

Want me to map out what the customer-facing chatbot would actually look like and what it could handle without ever touching a human?

Can the database query live website and other online resources when answering customer questions? Or would I have to screenshot those guides and add them to the database

Great question — and the answer is better than you might expect. You don't need screenshots.

Three ways to handle this, from simplest to most powerful

Option 1 — Scrape and store (recommended starting point)

You point a tool at your website, product guides, and documentation pages and it

You point a tool at your website, product guides, and documentation pages and it automatically pulls the text content into your `knowledge_articles` table. No screenshots, no manual copying. Clean readable text that Claude can query instantly.

Tools that do this: Firecrawl, Apify, or even a simple Python script. You run it once to seed the database, then schedule it to re-run weekly or whenever your site updates. If you update a troubleshooting guide on your website, the database stays current automatically.

This is the right approach for your own content — manuals, spec sheets, FAQ pages, troubleshooting guides, product documentation.

Option 2 — Live web search during a query

Instead of storing content, Claude searches the web in real time when answering a question. You actually already have this available — Exa is connected to your Claude account right now as an MCP tool. It's a web search API built specifically for AI use cases.

This is powerful for things that change frequently or live outside your control — supplier specs, industry standards, third-party component datasheets, forum discussions about CNC issues.

The tradeoff: it's slower than a database query (2-4 seconds) and you can't control what it finds. For a customer chatbot you'd want guardrails on what domains it's allowed to search.

Option 3 — Direct URL fetching

Claude can be given a specific URL and told to read it in real time. So if a customer asks about a specific machine and you have a dedicated page for it, Claude fetches that page fresh every time. Good for pages that update frequently like pricing or availability.

What I'd actually recommend for your setup

Use all three for different content types:

Content type	Approach
--------------	----------

Content type	Approach
Your own manuals, guides, FAQs	Scrape → store in knowledge_articles
Your product pages	Scrape weekly, keep database fresh
Fault codes and known fixes	Store directly — most reliable
Third-party component specs	Live web search via Exa
Industry forums and community fixes	Live web search via Exa
Competitor comparisons	Don't expose to chatbot — admin only

The screenshot question specifically

Screenshots are actually the worst option for this use case — Claude can read text in images, but it's slower, less accurate on technical content, and you can't search or update it easily. If your guides currently exist only as PDFs or image-heavy pages, the right move is to extract the text from them once and store it clean. There are tools that do this automatically including ones built right into Supabase's AI toolkit.

One thing worth knowing about Exa being connected

Since Exa is already connected to your Claude account, you could actually test this right now in this conversation. If you give me a URL from your website — a product page, a troubleshooting guide, anything — I can fetch it and show you exactly what Claude would be able to see and use. That gives you a real sense of what the chatbot would have access to before building anything.

Want to try that, or would you rather map out the full content ingestion plan for your database first?

I think the scrape and store option is safest. I would want to load this database with the cleanest and best data I have to get started. I have random word documents, screenshots, knowledge articles, call

transcripts, tickets, etc. Do all of those need to be converted into tables?

No — and this is an important distinction that will save you a lot of unnecessary work.

Tables are for structured data. Text is for everything else.

Your database has two fundamentally different types of content and they're stored differently:

Structured data — things with clear fields and values that you'd want to filter, sort, or aggregate. This goes in tables.

Unstructured content — documents, transcripts, articles, guides. This gets stored as text, but in a smarter way than just dumping it in a table cell.

Where your content actually belongs

What you have	How it's stored	Why
Call transcripts	Text column in <code>calls</code> table	Already has structure around it — date, customer, tech, duration
ZenDesk tickets	Text columns in <code>tickets</code> table	Same — structured wrapper around unstructured content
Word documents / guides	Vector store	Full documents, best searched semantically
Screenshots of guides	Extract text first, then vector store	Claude can read screenshots but text is cleaner
Knowledge articles	<code>knowledge_articles</code> table	Short, structured, reusable — table makes sense
Product manuals	Vector store	Too long for a table cell, needs semantic search

(PDFs)

Fault code lists	<code>knowledge_articles</code> table	Short, structured, highly reusable
Order history	<code>orders</code> table	Pure structured data

The vector store — this is the key concept

This is the thing most people don't know about and it changes everything about how Claude searches your content.

A regular database finds things by exact match or keyword. You search for "E-47" and it finds rows containing "E-47." Simple but limited.

A vector store understands *meaning*. A customer says "my Z-axis is making a grinding noise and won't move" — a vector search finds your troubleshooting guide about servo motor faults even if those exact words never appear in the document. It matches on semantic similarity, not keywords.

Supabase has this built in — it's called **pgvector** and it's included for free. You don't need a separate system.

Here's how it works in plain terms:

```
Your Word doc gets ingested
    ↓
Broken into chunks (~500 words each)
    ↓
Each chunk converted to a vector
(a mathematical representation of its meaning)
    ↓
Stored in Supabase alongside your regular tables
    ↓
Customer asks a question
    ↓
Question converted to same vector format
    ↓
Database finds the chunks with closest meaning
    ↓
```

The whole process happens in under a second.

On your screenshots specifically

Screenshots are the one format that needs a conversion step before ingestion. Claude can read text in images, but for a database you want clean extracted text. The good news is this is largely automated now:

- PDFs with text: extract directly, no effort
 - Word documents: extract directly, no effort
 - Screenshots of text: run through OCR first — tools like Textract or even Claude itself can do this in bulk
 - Screenshots of diagrams or machine schematics: these are worth keeping as images — Claude can reference them visually when answering questions about physical components
-

What "loading the database with your best data" actually looks like in practice

Rather than trying to load everything at once, I'd prioritize ruthlessly. The quality of your starting data matters more than the quantity.

Load first — highest signal:

- Resolved ZenDesk tickets with clear resolutions written out
- Call transcripts where the issue was definitively solved
- Any official troubleshooting guides you've written internally
- Fault code documentation
- Product manuals for your most-called-about machines

Load second:

- General product documentation
- FAQ pages from your website

- Word documents from your team

Load last or not at all:

- Tickets that were closed without resolution notes
 - Calls that were transferred and never logged
 - Outdated documentation superseded by newer versions
 - Spam and solicitation tickets
-

The cleanup question

This is worth being honest about. If your Word documents and knowledge articles are inconsistent, contradictory, or outdated — loading them as-is will make Claude's answers worse, not better. Garbage in, garbage out.

Before ingestion you'd want someone to do a quick pass on your internal documents and flag anything that's outdated or contradicts current practice. You don't need to rewrite everything — just tag what's trustworthy and what isn't, and only load the trustworthy content first.

The transcripts and tickets are actually your most valuable raw material precisely because they're real — real problems, real resolutions, real language your customers use. Even messy, they're more useful than a polished document that doesn't reflect how issues actually present themselves on a call.

Bottom line

You're not converting everything to tables. You're building two complementary stores:

Structured tables — customers, machines, orders, tickets, calls. Clean rows and columns. Fast lookups.

Vector store — everything else. Documents, transcripts, guides, manuals. Searched by meaning.

Claude draws from both simultaneously when answering a question. The table tells it Randy has a Crossfire Pro XR under warranty with 3 prior calls. The vector store finds the most relevant troubleshooting content for his specific symptoms. Together they

the most relevant troubleshooting content for his specific symptom. Together they give an answer neither could produce alone.

Want me to put together a data audit checklist — a practical document your team could use to go through what you have, rate its quality, and decide what gets loaded in which phase?

Nice. Ok, where do we begin and what would be the phases and steps to roll this out. Stay high level right now

Here's the full roadmap. Everything we've discussed, sequenced in the right order.

Created a file, read a file

Click each phase to expand it. A few things worth highlighting at the high level:

Phase 0 costs nothing and starts this week. Export your RingCentral transcripts, export your ZenDesk tickets, drop them into a Claude Project. You'll learn more about your support operation in one afternoon than months of gut feel. That analysis also directly informs what the database needs to prioritize — so it's not just a quick win, it's foundational research for everything that follows.

Each phase delivers real value before the next one begins. You're not waiting 6 months to see ROI. Phase 1 gives you a searchable knowledge base. Phase 2 eliminates manual ticket routing. Phase 3 cuts handle times. Every phase stands on its own.

The phone decision doesn't block anything. The RingCentral vs Vapi question sits in Phase 3 where it belongs. Phases 0, 1, and 2 are completely independent of it — you build the brain first, then connect the phone to it later.

Humans stay in control throughout. Nothing goes autonomous until you've seen it work in assisted mode for 30+ days and made a conscious decision to extend its authority. That's intentional — your rural customer base and your team's trust both require it.

The honest total timeline is 4-6 months to get through all 5 phases, but you'll have meaningful wins visible to your team within the first 2-3 weeks. That matters for keeping momentum and internal buy-in.

What would you like to go deeper on first?

I know it's far out in the plan. But if we move away from RingCentral, can I take the number to Vapi?

Yes — this is called **number porting** and it's a standard telecom process. Your phone numbers are yours, not RingCentral's. They're legally required to release them.

The basics of how it works:

You submit a port request to Vapi (or whatever carrier sits under Vapi), provide proof of ownership — usually a recent phone bill showing the number — and RingCentral has to release it. The number goes dark for anywhere from a few minutes to a few hours during the actual cutover, which is why you schedule it on a weekend or after hours.

Vapi uses Twilio under the hood for most carrier functions, and Twilio has a mature porting process. It's not complicated — it's just paperwork and scheduling.

A few practical things to know

RingCentral won't make it easy. They're not going to sabotage it — they're legally prevented from doing that — but don't expect them to be enthusiastic. Retention calls, offers, delays in sending the authorization code. Just be persistent. This is normal with any carrier departure.

You'll need your account number and a PIN from RingCentral to authorize the release. Find those before you start the process — they can be buried in your account settings.

Port each number separately or as a batch. If you have multiple queue numbers you want to bring over, most carriers let you batch port them together in one request.

Keep RingCentral active until the port completes. Never cancel first — that can actually make porting harder or impossible on some numbers.

The short answer: yes, your numbers come with you. It's one of the few consumer protections in telecom that actually works in your favor.

Our troubleshooting guides have a lot of photos. Would the database be able to store those photos from the guides and know when to provide the right one

Yes — and this is actually a really important use case for CNC support specifically, because a photo of "loosen this screw" is worth ten paragraphs of text to a customer standing in front of a machine.

Here's how it works:

How images get stored and retrieved

Images themselves don't go in the database as raw files — that would be slow and expensive. Instead you use a two-part system:

The image file lives in object storage — Supabase has this built in, it's called Supabase Storage. Think of it like a private Google Drive that your database controls. Each image gets a URL.

The metadata and context lives in the database as text — what machine it's for, what step it illustrates, what fault code it relates to, what the surrounding text says. This is what Claude actually searches.

```
Troubleshooting guide ingested
```

```
↓
```

```
Text chunks → vector store (searchable by meaning)
```

```
Images → Supabase Storage (stored with a URL)
```

```
Image context → database (what this image shows, where it appears)
```

```
↓
```

```
Customer asks about Z-axis alignment
```

```
↓
```

```
Claude finds the relevant text chunk
```

text chunk has an associated image URL
Claude returns the answer AND the image

The context around an image is everything

This is the key insight. Claude doesn't search images directly — it searches the text that describes and surrounds them. So when you ingest a guide, you want each image to have rich context stored alongside it:

- The heading it appears under
- The step number it belongs to
- A description of what the image shows
- The fault codes or machine models it relates to
- The caption if one exists

If your Word docs have well-captioned images with clear headings, most of this gets extracted automatically. If images are poorly labeled — just dropped into a document with no caption — a one-time human pass to add descriptions pays off enormously.

What Claude can do with a retrieved image

When the right image is found, Claude can do more than just return it. Because Claude is multimodal it can actually:

- Look at the image and describe what it shows in plain language
- Answer questions about what's depicted — "which screw is the one circled in red?"
- Combine the image with text context to give a richer answer than either alone

For a customer on your website chatbot asking "how do I re-tram my Crossfire Pro XR" — Claude returns the step-by-step text AND pulls the relevant photos inline. Same experience as your best tech walking them through it.

One honest limitation

If your guides have photos that are *only* photos with no surrounding text context — like a standalone image of a circuit board with no labels or captions — those are hard to retrieve reliably. Claude can describe what it sees in the image, but it has no way to know when to surface it unless there's text context telling the system what that image is for.

The fix is simple but requires a one-time effort: go through your guides and make sure every image has at minimum a caption and sits under a clear heading. That context is what makes the retrieval accurate.

The chatbot experience this enables

Done well, a customer on your website could have an exchange like:

Customer: My Crossfire Pro XR is showing an E-47 and the Z-axis won't move

Chatbot: E-47 is a Z-axis servo fault. Here are the steps to diagnose it:

Step 1: Check the LED on your servo driver — it should be solid green during normal operation.

(image: servo driver location on machine)

Step 2: If the LED is blinking once per second, the driver is in fault protection mode. Check the encoder cable connection shown here.

(image: encoder cable connection point)

That experience — text and images together, contextually accurate — is what separates a genuinely useful support tool from a generic chatbot that just talks at people.

The infrastructure for this is all within Supabase — Storage for the images, pgvector for the semantic search, and standard tables for the metadata. No separate system needed. Want me to add image handling as a specific step in the Phase 1 database build?

Another database question: historical order and new orders. I can import historical orders and warranties. How would the database maintain that info from epicor on an ongoing basis? Like a once a night sync?

Great question — and Epicor is actually a well-known ERP in the manufacturing space so this is a solved problem.

The short answer

Yes, a nightly sync is exactly the right approach for order and warranty data. Real-time sync is overkill for support purposes — if a customer calls, knowing their order status as of last night is almost always good enough. The exception would be order status checks like "where is my shipment" — but even that can be handled with a direct Epicor lookup at call time if needed.

How Epicor connects to your database

Epicor has several ways to get data out:

Epicor REST API — the cleanest modern option. Epicor 10 and Kinetic both have a built-in REST API. A developer writes a script that calls it nightly, pulls new and updated orders, and upserts them into your Supabase database. Upsert means insert if new, update if changed — so you're never duplicating records.

Epicor BAQ (Business Activity Queries) — Epicor's built-in reporting layer. You build a query in Epicor that returns exactly the fields you want, expose it as a REST endpoint, and your sync script calls it. Many Epicor admins are comfortable with BAQs even without deep development experience.

Direct database connection — Epicor runs on SQL Server underneath. A developer can connect directly to the Epicor SQL database and query it. Fast and flexible but requires your IT team to allow that connection, which some companies are uncomfortable with for security reasons.

Scheduled export — simplest but least elegant. Epicor generates a CSV or XML export on a schedule, drops it to a folder or FTP, and your sync script picks it up. No API needed. Works well if the API route hits bureaucratic resistance internally.

What a nightly sync actually looks like

11:00 PM – Sync script runs automatically

↓

Calls Epicor API – pulls all orders
modified in last 24 hours

↓

For each order:

- Does customer exist in Supabase?
If not → create customer record
- Does machine exist?
If not → create machine record
- Upsert order with latest status
- Recalculate warranty status
- Flag any warranty expirations
in next 30 days

↓

12:00 AM – Sync complete, log written

↓

Any errors → alert sent to admin

The whole thing runs unattended. You wake up in the morning and your database reflects last night's orders.

Warranty status specifically

This is worth handling smartly rather than just storing a date. Rather than just syncing a `warranty_expires` field and leaving it at that, you'd have the sync script automatically calculate and update a few derived fields every night:

- `in_warranty` — true or false right now
- `warranty_days_remaining` — integer, useful for prioritizing calls
- `warranty_expiring_soon` — flag for machines expiring within 30 days

That last one is actually a proactive opportunity most companies miss. You could automatically trigger a ZenDesk ticket or an outbound call to customers whose warranty is expiring in 30 days — either to sell an extended warranty or just to check in. That's a revenue and retention play sitting inside your support database.

The Epicor-specific question to ask your team

The main variable I can't tell you from here is which version of Epicor you're running and what API access has already been configured. Epicor Kinetic (the modern cloud version) has excellent REST APIs out of the box. Older on-premise Epicor 9 or 10

installations vary — some have the API fully enabled, some need an IT ticket to turn it on.

Before a developer starts building the sync, someone needs to answer:

- What version of Epicor are you on?
- Is the REST API currently enabled?
- Is Epicor on-premise or cloud hosted?
- Who is your Epicor admin internally?

Those four answers determine whether this is a 2-day job or a 2-week job. If the API is already enabled and accessible, this is straightforward. If it's locked down on-premise with a skeptical IT department, there's a conversation to have first.

One thing worth adding to the sync

Every time the sync runs, you'd also want it to cross-reference against your support history. If a machine has had 3 or more support tickets in the last 60 days, flag it. If a customer has an open ticket AND their warranty expires in 2 weeks, flag it. These are the kinds of insights that only exist when your order data and support data live in the same place — and a nightly sync is what makes that possible.

What version of Epicor are you running? That'll tell us a lot about how straightforward the integration will be.

Kinetic, cloud. Im admin and I'm very strong with epicor

That's the best possible answer. Kinetic cloud with you as admin makes this one of the easier integrations on the whole roadmap.

Why this is good news

Epicor Kinetic cloud has a fully modern REST API enabled by default — no IT tickets, no on-premise firewall conversations, no SQL Server access requests. You're already admin so you can generate API credentials yourself without waiting on anyone.

And because you know Epicor deeply, you'll know exactly which tables and BAQs have the cleanest data. That matters more than most people realize — ERPs accumulate messy data over years and someone who knows the system knows where to trust it and where not to.

How you'd set up the connection

Step 1 — Generate API credentials in Epicor

In Kinetic cloud you create an API key directly in the admin panel:

- Security → API Keys → Generate New Key
- Scope it to read-only — your sync script never needs to write back to Epicor
- Note the key, your company ID, and your Kinetic cloud URL

That's all the developer needs from the Epicor side.

Step 2 — Build or identify your BAQs

This is where your Epicor strength becomes a real advantage. Rather than a developer trying to reverse-engineer your data model, you can build the BAQs yourself and hand them over as clean endpoints. The developer just calls them.

The BAQs you'd want:

BAQ	What it returns	Sync frequency
Orders modified since date	All orders updated in last 24hrs	Nightly
Order line items	Parts, quantities, prices per order	Nightly

Customer master	Name, company, contact, address	Nightly
Shipped machines	Serial number, model, ship date, customer	Nightly
Warranty records	Serial, warranty start, warranty end, type	Nightly
Open orders	Anything not yet fulfilled	Nightly

Each BAQ gets exposed as a REST endpoint in Kinetic. Your developer calls each one, maps the fields to your Supabase schema, and the sync is done.

Step 3 — The sync script

A relatively simple Python or Node.js script that runs on a schedule — either on a cheap cloud server or a service like Railway or Render for about \$5/month. It:

- Calls each BAQ endpoint
- Transforms the Epicor field names to your Supabase column names
- Upserts records — inserts new, updates changed
- Logs what ran and flags any errors
- Takes about 15-30 minutes to write once the BAQs are ready

The field mapping conversation

This is where you and the developer need to sit together for about an hour. Epicor has its own naming conventions — `CustNum`, `OrderNum`, `ShipToNum`, `PartNum` — and your Supabase schema uses readable names like `customer_id`, `order_number`, `machine_serial`. Someone has to map one to the other.

Because you know Epicor, you can probably produce that mapping in a spreadsheet faster than a developer could figure it out themselves. Something like:

Epicor field	Epicor table	Supabase table	Supabase column
CustNum	Customer	customers	epicor_customer_id
Name	Customer	customers	company
OrderNum	OrderHed	orders	order_number

OrderDate	OrderHed	orders	order_date
SerialNumber	SerialNo	machines	serial_number
WarrantyExpDate	SerialWarranty	machines	warranty_expires

That spreadsheet is one of the most valuable things you can produce before development starts — it probably saves a week of back and forth.

One thing Kinetic cloud gives you that on-premise doesn't

Webhooks. Kinetic cloud can push events outward in real time when records change — a new order is placed, a warranty is updated, a shipment goes out. That means for certain high-value events you don't have to wait for the nightly sync. A machine ships today, your database knows today, and if that customer calls this afternoon your tech already sees it.

For most data a nightly sync is fine. But for shipment confirmations and new machine registrations, real-time webhooks are worth setting up — they're available in Kinetic and they're not much extra work once the nightly sync is already built.

Honest timeline for this piece

Given Kinetic cloud, your admin access, and your ability to build the BAQs yourself:

- BAQs built and tested by you: 1-2 days
- Field mapping spreadsheet: a few hours
- Developer builds sync script: 1-2 days
- Testing and validation: 1 day

This is realistically a one-week effort, most of which is you doing Epicor work you already know how to do. The developer's part is almost mechanical once you hand them clean BAQ endpoints and a field mapping doc.

This might actually be the fastest win on the entire roadmap once Phase 1 database is standing — because so much of the heavy lifting is in your hands rather than dependent on third parties